

Unit testing



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

1

Core Module

- Continuous integration (CI) ensures that codebase updates are automatically tested.
- CI helps catch bugs early but does not guarantee a bug-free codebase.
- Tests executed in CI are crucial for its effectiveness.

Martin Fowler:

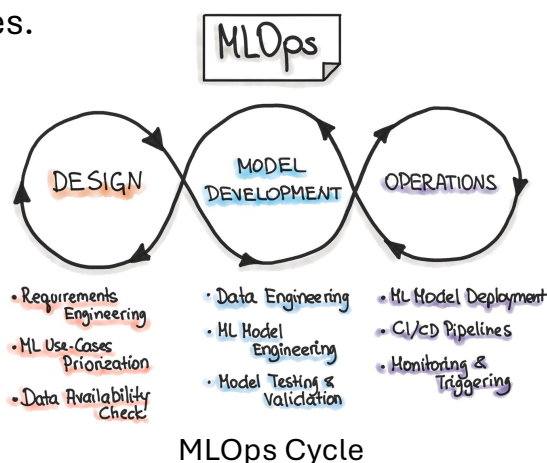
Continuous Integration doesn't get rid of bugs, but it does make them dramatically easier to find and remove.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

- Continuous integration is a cornerstone of the MLOps cycle.
- CI structures and automates testing but is only as strong as the tests themselves.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

Unit Testing

- Unit testing focuses on testing individual parts of the code (functions, modules, etc.).
- Allows easy isolation of the part of code that breaks after updates.
- Different from integration testing, which is equally important but not the focus here.

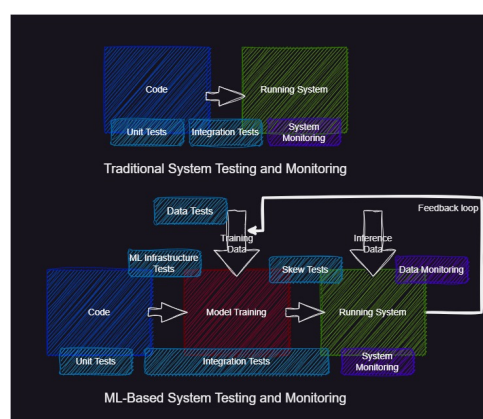


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

Unit Testing in MLOps

- Unit tests are crucial for machine learning systems.
- Machine learning systems depend on data, adding complexity.
- Testing must include:
 - Unit tests
 - Integration tests
 - Data testing
 - Infrastructure testing
 - Monitoring data distribution



System Complexity



Pytest Overview

- Pytest is a framework used to write and run tests in Python.
- Allows easy execution of tests and provides detailed test output.

Exercises

1. Create a `tests` folder.
2. Read the [Pytest Getting Started Guide](#).

3. Install Pytest:

```
bash      pip install pytest
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Writing Tests

- Create test files named `test_*.py`.
- Wrap each test in a function starting with `test_*`:

```
# this will be found and executed by pytest
```

```
def test_something():
```

```
    ...
```

```
# this will not be found and executed by pytest
```

```
def something_to_test():
```

```
    ...
```

4. Check if tests are passing: `bash pytest tests/`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

Data Testing

- Create tests/test_data.py.

- Example test structure:

```
def test_data():
    dataset = MNIST(...)
    assert len(dataset) == N_train for training and
N_test for test
    assert that each datapoint has shape [1,28,28]
or [784] depending on how you choose to format
    assert that all labels are represented
```

- Ensure data points and labels are correct.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

Data Testing - Solution

- Solution

```
from my_project.data import corrupt_mnist

def test_data():
    train, test = corrupt_mnist()
    assert len(train) == 30000
    assert len(test) == 5000
    for dataset in [train, test]:
        for x, y in dataset:
            assert x.shape == (1, 28, 28)
            assert y in range(10)
    train_targets = torch.unique(train.tensors[1])
    assert (train_targets == torch.arange(0,10)).all()
    test_targets = torch.unique(test.tensors[1])
    assert (test_targets == torch.arange(0,10)).all()
```

- Ensure data points and labels are correct.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

Model Testing

- Create tests/test_model.py.

- Example test structure:

```
def test_model():
    model = MyAwesomeModel()
    x = torch.randn(1, 1, 28, 28)
    y = model(x)
    assert y.shape == (1, 10)  # Check model output
    shape
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

Model Testing

- More example

```
# src/models/model.py
def forward(self, x: Tensor):
    if x.ndim != 4:
        raise ValueError('Expected input to a 4D tensor')
    if x.shape[1] != 1 or x.shape[2] != 28 or x.shape[3] != 28:
        raise ValueError('Expected each sample to have shape [1, 28, 28]')
```

- Better one

```
# tests/test_model.py
import pytest
from my_project.model import MyAwesomeModel

def test_error_on_wrong_shape():
    model = MyAwesomeModel()
    with pytest.raises(ValueError, match='Expected input to a 4D tensor'):
        model(torch.randn(1, 2, 3))
    with pytest.raises(ValueError, match='Expected each sample to have shape [1, 28, 28]'):
        model(torch.randn(1, 1, 28, 29))
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

Training Testing

- Create tests/test_training.py.
- Example: assert proper training behavior, raise warnings/errors where needed.

```
def test_training():
    ...
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

13

Coverage and Parametrization

- Use `pytest.mark.parametrize` to test different input arguments:

```
@pytest.mark.parametrize("batch_size", [32, 64])
def test_model(batch_size: int):
    model = MyModel()
    x = torch.randn(batch_size, 1, 28, 28)
    y = model(x)
    assert y.shape == (batch_size, 10)
```

- Use coverage to measure how much of your codebase is being tested:

```
pip install coverage
coverage run -m pytest tests/
coverage report
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

14

Knowledge Check

1. Does 100% code coverage ensure no bugs?
 - No, it just ensures all lines are executed, but it doesn't guarantee the absence of bugs.
2. How many tests run based on parametrize settings?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

15

Additional Tools

- **unittest**: Python's built-in testing framework.
- **hypothesis**: Helps find corner case errors.
- **doctest**: Tests code examples included in documentation.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

16